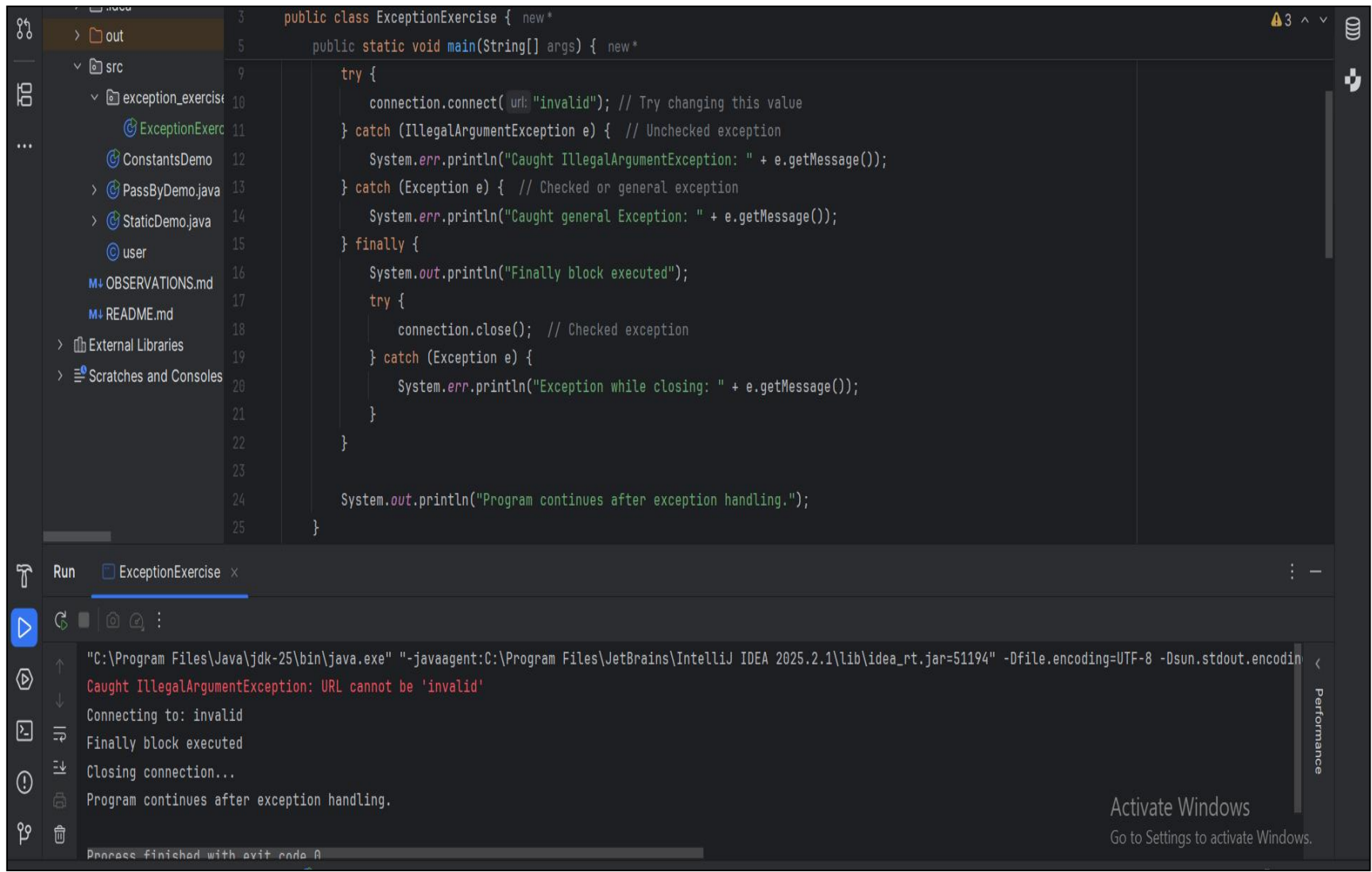


- 1 its catch the first block “IllegalArgumentException”, because the thrown exception matches it .



The screenshot displays the IntelliJ IDEA IDE. The left sidebar shows a project structure with a package named `exception_exercise` containing several Java files, including `ExceptionExercise.java`. The main editor window shows the source code of `ExceptionExercise`. The code defines a `main` method that uses a `try-catch-finally` block to handle exceptions. It attempts to connect to an invalid URL, which triggers an `IllegalArgumentException`. The first catch block handles this specific exception, while the second catch block handles any other `Exception`. The `finally` block ensures that the connection is closed and a message is printed. The output window at the bottom shows the execution results, confirming that the `IllegalArgumentException` was caught and handled correctly.

```
public class ExceptionExercise {  
    public static void main(String[] args) {  
        try {  
            connection.connect(url: "invalid"); // Try changing this value  
        } catch (IllegalArgumentException e) { // Unchecked exception  
            System.err.println("Caught IllegalArgumentException: " + e.getMessage());  
        } catch (Exception e) { // Checked or general exception  
            System.err.println("Caught general Exception: " + e.getMessage());  
        } finally {  
            System.out.println("Finally block executed");  
            try {  
                connection.close(); // Checked exception  
            } catch (Exception e) {  
                System.err.println("Exception while closing: " + e.getMessage());  
            }  
        }  
        System.out.println("Program continues after exception handling.");  
    }  
}
```

Run ExceptionExercise x

"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.1\lib\idea_rt.jar=51194" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8

Caught IllegalArgumentException: URL cannot be 'invalid'

Connecting to: invalid

Finally block executed

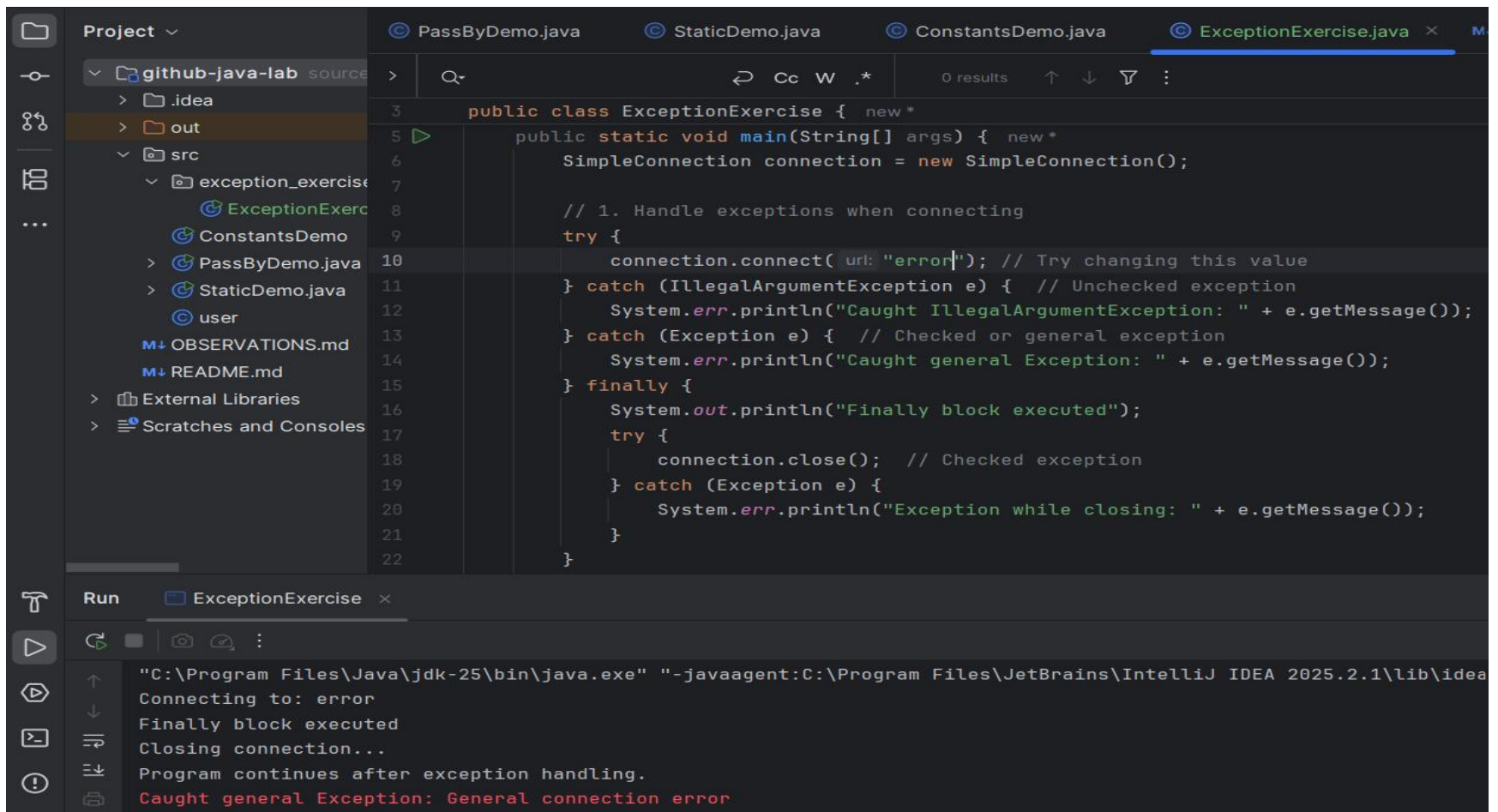
Closing connection...

Program continues after exception handling.

Process finished with exit code 0

Activate Windows
Go to Settings to activate Windows.

- 2 caught by the second catch , because it's a checked exception .



```
Project ▾
  ▾ github-java-lab source
    > .idea
    > out
    ▾ src
      ▾ exception_exercise
        ExceptionExercise.java
      ConstantsDemo.java
      PassByDemo.java
      StaticDemo.java
      user
    OBSERVATIONS.md
    README.md
  External Libraries
  Scratches and Consoles

PassByDemo.java StaticDemo.java ConstantsDemo.java ExceptionExercise.java x M
3 public class ExceptionExercise { new *
5   public static void main(String[] args) { new *
6     SimpleConnection connection = new SimpleConnection();
7
8     // 1. Handle exceptions when connecting
9     try {
10      connection.connect( url: "error"); // Try changing this value
11    } catch (IllegalArgumentException e) { // Unchecked exception
12      System.err.println("Caught IllegalArgumentException: " + e.getMessage());
13    } catch (Exception e) { // Checked or general exception
14      System.err.println("Caught general Exception: " + e.getMessage());
15    } finally {
16      System.out.println("Finally block executed");
17      try {
18        connection.close(); // Checked exception
19      } catch (Exception e) {
20        System.err.println("Exception while closing: " + e.getMessage());
21      }
22    }
23  }
```

Run ExceptionExercise x

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.1\lib\idea
Connecting to: error
Finally block executed
Closing connection...
Program continues after exception handling.
Caught general Exception: General connection error
```

- 3 there is no exception , normal flow .

The screenshot displays the IntelliJ IDEA IDE with the 'ExceptionExercise.java' file open. The code defines a `SimpleConnection` class and a `main` method that demonstrates exception handling. The program successfully connects to 'abc', prints 'Connection successful', and then prints 'Finally block executed' and 'Closing connection...' before continuing with 'Program continues after exception handling.'.

```
3 public class ExceptionExercise { new *
4
5     public static void main(String[] args) { new *
6         SimpleConnection connection = new SimpleConnection();
7
8         // 1. Handle exceptions when connecting
9         try {
10             connection.connect( url: "abc"); // Try changing this value
11         } catch (IllegalArgumentException e) { // Unchecked exception
12             System.err.println("Caught IllegalArgumentException: " + e.getMessage());
13         } catch (Exception e) { // Checked or general exception
14             System.err.println("Caught general Exception: " + e.getMessage());
15         } finally {
16             System.out.println("Finally block executed");
17             try {
18                 connection.close(); // Checked exception
19             } catch (Exception e) {
20                 System.err.println("Exception while closing: " + e.getMessage());
21             }
22         }
23     }
```

Run ExceptionExercise x

"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.1\lib\idea
Connecting to: abc
Connection successful
Finally block executed
Closing connection...
Program continues after exception handling.

Experiment

- 1 when we switch the exception its give an error. 2nd block become unreachable .

The screenshot shows an IDE with a project named 'github-java-lab'. The file 'ExceptionExercise.java' is open, showing the following code:

```
package src.exception_exercise;

public class ExceptionExercise { new *

    public static void main(String[] args) { new *
        SimpleConnection connection = new SimpleConnection();

        // 1. Handle exceptions when connecting
        try {
            connection.connect( url: "abc"); // Try changing this value
        } catch (Exception e) { // Unchecked exception
            System.err.println("Caught general Exception: " + e.getMessage());
        } catch (IllegalArgumentException e) { // Checked or general exception
            System.err.println("Caught IllegalArgumentException: " + e.getMessage());
        } finally {
            System.out.println("Finally block executed");
            try {
                connection.close(); // Checked exception
            } catch (Exception e) {
                System.err.println("Exception while closing: " + e.getMessage());
            }
        }
    }
}
```

The IDE shows four compiler errors in the 'Problems' panel:

- Exception 'java.lang.IllegalArgumentException' has already been caught :13
- Modifier 'public' is redundant for 'main' method on Java 25 :5
- Parameter 'args' is never used :5
- Exception 'java.lang.Exception' is never thrown in the method :46

- In all input value hits the new exception that I added .

The screenshot shows an IDE with the following components:

- Project Explorer:** Shows a project named 'github-java-lab' with folders '.idea', 'out', and 'src'. Inside 'src', there is a folder 'exception_exercise' containing 'ExceptionExercise.java', 'ConstantsDemo.java', 'PassByDemo.java', 'StaticDemo.java', and 'user'. There are also files 'OBSERVATIONS.md' and 'README.md'.
- Code Editor:** Displays the code for 'ExceptionExercise.java'. The code defines a public class 'ExceptionExercise' with a static class 'SimpleConnection' that implements 'AutoCloseable'. It includes methods 'connect', 'close', and 'NetWorkFailure'.
- Run Console:** Shows the output of the program. It starts with 'NetWorkFailure', followed by 'Program continues after exception handling.', and then two lines of caught exceptions: 'Caught general Exception: General connection error' and 'Caught network exception: NetWorkFailure'.

```
public class ExceptionExercise {  
    static class SimpleConnection implements AutoCloseable {  
        public void connect(String url) throws Exception {  
        }  
        @Override  
        public void close() throws Exception {  
            System.out.println("Closing connection...");  
        }  
        public void NetWorkFailure() throws Exception {  
            System.out.println("NetWorkFailure");  
            throw new Exception("NetWorkFailure");  
        }  
    }  
}
```

Run ExceptionExercise x

NetWorkFailure
Program continues after exception handling.
Caught general Exception: General connection error
Caught network exception: NetWorkFailure

throw vs throws

- Throws : we use it when the method we added we know that its might throw an exception , and we added it in the top of the method “ throws <exception/type of exception”.
- Throw : we use it inside the method , and actually throwing an exception object , we can add an error text “ throw new exception (“error”)”.

Exception class and hierarchy

- Exception is a subclass of throwable , and runtime exception is a subclass of exception , every runtime exception is an exception , but not all exception are a runtime exception .
- Runtime exception are unchecked exception like (NullPointerException,ClassCastException,..), exceptions like (IOException,SQLException,...), are checked exception.

Checked vs. Unchecked Exceptions

- Checked exception : are checked at compiling file and our project wont compile before we fix the error , ex (IOException).
- Unchecked exception : they are a runtime exception, ex(IllegalArgumentException).

finally block

- It's a part of code that always execute after a try block .
- We use it to close connection in the end .

Order of catch blocks

- Because if we put a general exception in the top the specific one can not be reached ,so we should put the specific exceptions first .
- Because the “exception e” is a general exception and the “IllegalArgumentException e” is a specific one.